OligoMet Profiler: In-Silico Metabolite Identification for Therapeutic Oligonucleotides

true

2026-08-29

Abstract

OligoMetProfiler generates theoretical metabolite libraries, molecular formulas, negative-ESI charge envelopes, isotope patterns, phosphorothioate (PS) oxidation series, McLuckey MS/MS fragment ions, and PRM inclusion lists for any therapeutic oligonucleotide – DNA, RNA, 2'-O-methyl, 2'-fluoro, MOE, cEt, LNA, mixed PO/PS backbones, and fully custom chemistry via dictionary overrides. It optionally matches the theoretical library against acquired MS data (mzML/mzXML or peak lists), exports a seven-sheet Excel workbook and an HTML/PDF report, and writes targeted mass lists formatted for the Thermo Orbitrap Exploris Method Editor. This vignette walks through the complete workflow: sequence input in three notations, the chemistry dictionary and custom overrides, metabolite library generation, mass and envelope calculation, fragment ion prediction, acquisition method export, MS data matching, and the Shiny dashboard.

Introduction

Metabolite identification for therapeutic oligonucleotides (antisense oligonucleotides, siRNA, and related modalities) requires predicting the masses of every plausible biotransformation product before any of them can be found in LC-MS data. Exonucleases chew nucleotides from the 3' and 5' termini, endonucleases cleave internal linkages, phosphorothioate (PS) backbones progressively desulfurize to phosphodiester (PO) *in vivo* and *in vitro*, and each of these species ionizes as a multi-charge negative-ESI envelope with sodium, potassium, and ammonium adduct satellites. Doing this by hand in a spreadsheet is error-prone and does not scale past a handful of metabolites.

`OligoMetProfiler` automates the whole chain. Given a sequence and its chemistry (sugars, linkages, terminal conjugates), it produces:

- **Theoretical metabolite library** – parent, 3'/5' exonuclease truncation ladders, and endonuclease internal fragments.
- **Masses and formulas** – elemental formula, monoisotopic mass, and average mass for every metabolite, assembled from a validated chemistry dictionary.
- **Charge envelopes** – $[M - zH]^{z-}$ m/z values across a configurable charge-state range, with H/Na/K/NH_4 adduct variants.
- **Isotope patterns** – top-N isotope peaks per charge state, via `enviPat` when installed or a built-in convolution engine otherwise.
- **PS oxidation series** – the -15.977 Da PS-to-PO desulfurization ladder at each oxidation level.
- **McLuckey MS/MS fragment ions** – a, a-B, b, b-B, c, d, w, x, y, z terminal ions plus internal fragments, with PS diagnostic ions, following the nomenclature of McLuckey and co-workers.
- **Acquisition support** – a PRM inclusion list, plus Orbitrap Exploris Method Editor-formatted MS1 inclusion and MS2 PRM target lists.
- **Reporting** – a seven-sheet Excel workbook and an HTML/PDF report with summary plots.
- **MS matching (optional)** – import mzML/mzXML or peak lists, match MS1 features against the library with ppm tolerance, score isotope fit and envelope consistency, and confirm hits via MS/MS fragment matching.

The methodology is grounded in four published approaches: SynONIM (Lippens *et al.*, JASMS 2024) for synthetic oligonucleotide modification and impurity nomenclature, the eluforsen metabolite profiling study (Kim *et al.*, Mol Ther Nucleic Acids 2019) for the LC-MS/MS workflow and PS diagnostic ions, OligoDistiller (Liu *et al.*, Anal Chem 2025) for McLuckey fragment annotation and isotope envelope fitting, and the FMVS automatic metabolite identification method (Ye *et al.*, J Chromatogr B 2025) for confirmation scoring.

Four approved oligonucleotide therapeutics are added as working examples, one per modality class in Takakusa *et al.* (2023): **nusinersen** (antisense SSO), **inotersen** (antisense gapmer), **patisiran** (siRNA delivered by lipid nanoparticle) and **givosiran** (GalNAc-conjugated siRNA). The two antisense examples double as the formula engine's regression anchors – the pipeline reproduces their published free-acid molecular formulas exactly. They are illustrations, not default targets: every function in this vignette runs on any sequence you give it.

Installation

Install directly from GitHub:

```
# install.packages("remotes")
remotes::install_github("nishi76/OligoMet-Profiler")
```

The hard dependencies (`openxlsx`, `ggplot2`, `xml2`, `xfun`) are installed automatically. Two optional packages improve specific steps:

- `enviPat` – higher-accuracy isotope patterns (a built-in convolution engine is used when it is absent);
- `rmarkdown` – required only for rendering HTML/PDF reports.

The Shiny dashboard additionally uses `shiny`, `DT`, `bslib`, and `shinyFiles`; those are pulled in by `install_packages.R` in the repository root, or install them manually.

```
library(OligoMetProfiler)
```

Quick start

A complete run on a 16-mer gapmer ASO – 2'-O-methyl wings, DNA gap, all-PS backbone – takes four calls:

```
seq_str <- "Gm-sTm-sCm-sTm-sCm-sTd-sCd-sTd-sCd-sTd-sTd-sCm-sTm-sCm-sTm-sGm"

spec <- parse_input(seq_str)
mets <- generate_metabolites(spec, opts = list(
  oligo_name = "demo", max_3p = 5, max_5p = 5, endo = TRUE))

info <- metabolite_mass_info(mets[[1]])
cat("Parent formula:", info$formula_str,
    "\nMonoisotopic mass:", round(info$mono_mass, 4), "Da\n")
#> Parent formula: C164H221N44O97P15S15
#> Monoisotopic mass: 5302.559 Da

length(mets)   # parent + truncations + endonuclease fragments
#> [1] 37
```

The Excel workbook and HTML report wrap everything downstream of this into single calls (`build_workbook()`, `build_report()`); they are covered in the *Outputs* section below.

From analytical documentation to an input string

The pipeline needs three pieces of information *per nucleotide position*, plus anything attached to the ends. All of it is normally on the documentation that accompanies the oligonucleotide – a Certificate of Analysis (CoA), a synthesis report, a patent sequence listing, or a BioPharma Finder sequence-editor export:

1. **Base sequence** (5' to 3'). Watch for 5-methylcytosine: CoAs write it as `mC`, `5-Me-C`, `C*`, or a footnote such as "all cytosines are 5-methyl". Its base code here is `S`, not `C` – the two differ by CH_2 (14.016 Da) per site.
2. **Sugar at each position**, often given as a design shorthand: "5-10-5 MOE gapmer" means 5 MOE positions, 10 DNA, 5 MOE; "2'-OMe/2'-F alternating" alternates `m` and `f`.
3. **Backbone linkages** – "full phosphorothioate" is the common case for ASOs; mixed backbones are listed per position or as a pattern ("PS wings, PO gap"). An *n*-mer has *n - 1* linkages.
4. **Terminal conjugates** (GalNAc, cholesterol, amino linkers, phosphate caps, dyes) – absent unless stated.

To construct the triplet string, write one dash-separated token per position, 5' to 3', as `[incoming linkage][BASE][sugar]`, leaving the linkage prefix off the first token.

Take inotersen as a worked case. Its published description reads:

> 20-mer antisense oligonucleotide targeting the TTR 3'UTR (bases 618-637), sequence 5'-TCTTGGTTACATGAAATCCC-3'. A 5-10-5 gapmer: positions 1-5 and 16-20 are 2'-MOE, positions 6-15 are 2'-deoxy. Fully phosphorothioate backbone. All cytosines are 5-methylcytosine.

Translating position by position:

Position	Base (from CoA)	Base code	Sugar	Incoming linkage	Token
1	T	`T`	2'-MOE -> `e`	(none, 5' end)	`Te`
2	5-Me-C	`S`	2'-MOE -> `e`	PS -> `s`	`sSe`
6	G	`G`	DNA -> `d` (gap starts)	PS -> `s`	`sGd`
16	A	`A`	2'-MOE -> `e` (wing resumes)	PS -> `s`	`sAe`
20	5-Me-C	`S`	2'-MOE -> `e`	PS -> `s`	`sSe`

Joined up, that is the `INOTERSEN_TRIPLET` constant shipped with the package:

```
spec_coa <- parse_input(INOTERSEN_TRIPLET)
spec_coa$n
#> [1] 20
paste(spec_coa$bases, collapse = "")
#> [1] "TSTTGGTTASATGAAATSSS"
paste(spec_coa$sugars, collapse = "")
#> [1] "eeeeeddddddddddeeeee"
```

The proof that the translation is right is the molecular formula: the assembled formula matches the one on the approved product labelling exactly.

```
info_coa <- metabolite_mass_info(list(
  bases = spec_coa$bases, sugars = spec_coa$sugars,
  linkages = spec_coa$linkages, conj5 = "none", conj3 = "none"))
info_coa$formula_str
#> [1] "C230H318N69O121P19S19"
INOTERSEN_FORMULA            # published free-acid formula
#> [1] "C230H318N69O121P19S19"
```

```
round(info_coa$avg_mass, 2)  # published average MW: 7183.08
#> [1] 7183.08
```

Two traps are worth calling out, because both change the mass and neither is obvious from the letter sequence alone. Writing `C` instead of `S` for a 5-methylcytosine loses CH_2 (14.016 Da) *per site* – in inotersen that is five sites, a 70 Da error. And an n-mer has n - *1* linkages, so the last token never carries an outgoing bond.

The parser enforces the obvious sanity checks – token count equals oligo length, every token after the first starts with a linkage code, and any code missing from the dictionary is named in the error message. Modifications without a built-in code are added as custom overrides with their elemental formula (see *Custom chemistry overrides* below). The package's `inst/help/QUICKSTART.md` carries a condensed version of this guide with the full code tables.

The four bundled reference drugs

One approved therapeutic per modality class ships with the package, covering the chemistries you are most likely to meet:

```
do.call(rbind, lapply(REFERENCE_OLIGOS, function(r) data.frame(
  drug = r$name, brand = r$brand, modality = r$modality,
  nt = r$length, chemistry = r$chemistry, stringsAsFactors = FALSE)))
#>                                drug    brand           modality nt
#> nusinersen               nusinersen SPINRAZA    Antisense (SSO) 18
#> inotersen                 inotersen  TEGSEDI Antisense (gapmer) 20
#> patisiran  patisiran (sense strand) ONPATTRO        siRNA (LNP) 21
#> givosiran  givosiran (sense strand) GIVLAARI     siRNA (GalNAc) 21
#>                                                          chemistry
#> nusinersen                PS, uniform 2'-MOE, 5-methyl pyrimidines
#> inotersen           PS, 5-10-5 2'-MOE/DNA gapmer, 5-methylcytosine
#> patisiran  2'-OMe / ribose, phosphodiester backbone, dTdT overhang
#> givosiran           2'-OMe / 2'-F, partial PS, 3' trivalent GalNAc
```

Each is available as a triplet string (`NUSINERSEN_TRIPLET`, `INOTERSEN_TRIPLET`, `PATISIRAN_SENSE_TRIPLET`, `GIVOSIRAN_SENSE_TRIPLET`) and through the `REFERENCE_OLIGOS` registry, which also carries the terminal conjugates that triplet notation cannot express:

```
for (r in REFERENCE_OLIGOS) {
  sp <- parse_input(r$triplet)
  info <- metabolite_mass_info(list(
    bases = sp$bases, sugars = sp$sugars, linkages = sp$linkages,
    conj5 = r$conj5, conj3 = r$conj3))
  cat(sprintf("%-26s %-20s %2d nt  %-38s %10.4f Da\n",
              r$name, r$modality, sp$n, info$formula_str, info$mono_mass))
}
#> nusinersen                 Antisense (SSO)      18 nt  C234H340N610O128P17S17                  7122.
#> inotersen                  Antisense (gapmer)   20 nt  C230H318N690O121P19S19                  7178.
#> patisiran (sense strand)   siRNA (LNP)          21 nt  C210H268N76O143P20                     6761.
#> givosiran (sense strand)   siRNA (GalNAc)      21 nt  C247H324N82O160P20S2F5                  7776.
```

Note what distinguishes each modality, since these are exactly the features that drive metabolite prediction:

- **nusinersen** – uniform 2'-MOE and a fully PS backbone. Sugar-modified throughout, so it degrades by exonuclease nibbling from the termini rather than internal cleavage.
- **inotersen** – a 5-10-5 gapmer, MOE wings around a DNA gap. The gap is the endonuclease-susceptible region, so `endo_sites = "gap"` matches its real biotransformation route.

- **patisiran** (sense strand) – an all-phosphodiester siRNA with a dTdT overhang. No PS means no oxidation series to model (`max_oxid` has nothing to act on).
- **givosiran** (sense strand) – alternating 2'-OMe/2'-F, PS only at the two 5'-terminal linkages, and a trivalent GalNAc at the 3' end. The conjugate rides on the parent and on 5' truncations, but is lost from 3' truncations that cut past it.

Duplex drugs are given as their sense strand: this pipeline profiles one strand at a time, so run each strand of an siRNA separately.

Sequence input notations

`parse_input()` auto-detects three input formats. All three reduce to the same internal representation: parallel vectors of bases, sugars, and linkages (5' to 3'), plus optional 5'/3' conjugate codes.

Triplet notation

Each dash-separated token is `[linkage][base][sugar]`. The first (5') token has no linkage prefix; the prefix on token i is the incoming bond from position i - *1*.

```
spec <- parse_input(
  "Gm-sTm-sCm-sTm-sCm-sTd-sCd-sTd-sCd-sTd-sTd-sCm-sTm-sCm-sTm-sGm")
spec$n
#> [1] 16
paste(spec$bases, collapse = "")
#> [1] "GTCTCTCTCTTCTCTG"
paste(spec$sugars, collapse = "")
#> [1] "mmmmmddddddmmmmm"
```

Breaking the example down:

- `Gm` – position 1: base G, sugar `m` (2'-OMe), no incoming linkage.
- `sTm` – position 2: base T, sugar `m`, PS bond from position 1.
- `sTd` – position 6: base T, sugar `d` (DNA), PS bond from position 5.

Linkage codes are `o` or `p` for phosphodiester and `s` for phosphorothioate. `p` matches Thermo BioPharma Finder's sequence-editor notation, so triplet strings copied from BPF (e.g. `Ad-pTd-pCd-pAd`) parse directly.

OligoDistiller notation

Wrapped in `OH-...-OH`, with `*` marking a PS *outgoing* bond and the sugar as a suffix on the base:

```
spec_od <- parse_input(
  "OH-Gm*-Tm*-Cm*-Tm*-Cm*-Td*-Cd*-Td*-Cd*-Td*-Td*-Cm*-Tm*-Cm*-Tm*-Gm-OH")
identical(spec_od$bases, spec$bases)
#> [1] TRUE
```

Sugar suffixes: `m` = 2'-OMe, `f` = 2'-F, `d` = DNA, `r` = RNA, `e` = MOE.

Three-line notation (bases / sugars / linkages)

The layout a chemical analysis file or a BioPharma Finder sequence entry gives you directly, and the one the dashboard's **Manual sequence entry** panel uses. One code per position for bases and sugars; one per *bond* for linkages, so an n-mer has n-1 of them.

```
spec_3l <- parse_three_line(
  bases    = "TSASTTTSATAATGSTGG",
  sugars   = "eeeeeeeeeeeeeeeeee",
```

```
  linkages = "sssssssssssssssss")
format_triplet(spec_3l)
#> [1] "Te-sSe-sAe-sSe-sTe-sTe-sTe-sSe-sAe-sTe-sAe-sAe-sTe-sGe-sSe-sTe-sGe-sGe"
```

A field holding a single code is applied to every position, which is the common case for a uniformly modified drug – `sugars = "e"` and `linkages = "s"` give exactly the result above. Multi-character codes (`MOE`, `cEt`, `LNA`) need a separator: `"MOE-MOE-d-d"`, `"A,G,S,T"`.

`format_three_line()` renders any spec back to the three lines, and `format_biopharma_fasta()` writes it as a FASTA record for BioPharma Finder's Sequence Manager:

```
cat(format_biopharma_fasta(spec_3l, name = "nusinersen"))
#> >nusinersen | 18-mer
#> Te-sSe-sAe-sSe-sTe-sTe-sTe-sSe-sAe-sTe-sAe-sAe-sTe-sGe-sSe-sTe-sGe-sGe
```

See `inst/help/SEQUENCE_GUIDE.md` (or the dashboard's Help panel) for a walkthrough aimed at readers who have not worked with oligonucleotide notation before.

Structured input

An R list with explicit parallel vectors – use this when you need terminal conjugates or full programmatic control. `linkages[i]` is the bond from position i to $i + 1$; the last entry is `NA` (no outgoing bond at the 3' terminus).

```
my_spec <- parse_input(list(
  bases    = c("A", "C", "G", "U", "A", "G", "U", "C", "A", "C", "G", "U"),
  sugars   = c("m", "f", "m", "f", "m", "f", "m", "f", "m", "f", "m", "f"),
  linkages = c("s", "s", "s", "s", "s", "s", "s", "s", "s", "s", "s", NA),
  conj5    = "none",
  conj3    = "GalNAc"
))
my_spec$conj3
#> [1] "GalNAc"
```

`validate_spec()` checks any parsed spec against the dictionary and reports unknown codes before you get halfway through a run.

The chemistry dictionary

Every module resolves base, sugar, linkage, and conjugate codes through a chemistry dictionary. `STANDARD_DICT` is the built-in default:

```
# A couple of representative entries
STANDARD_DICT[["m"]]$name
#> [1] "2'-O-methylribose"
STANDARD_DICT[["s"]]$name
#> [1] "phosphorothioate (PS) -- also BioPharma Finder notation"
format_formula(STANDARD_DICT[["G"]]$formula)
#> [1] "C5H5N5O"
```

Built-in codes

Bases: `A` (adenine), `G` (guanine), `C` (cytosine), `T` (thymine), `U` (uracil), `S` (5-methylcytosine), `D` (2,6-diaminopurine), `I` (hypoxanthine/inosine).

Sugars: `d` (2'-deoxyribose), `r` (ribose), `m` (2'-O-methylribose), `f` (2'-fluororibose), `FANA` (2'-F arabino; isobaric with `f`), `e`/`MOE` (2'-O-methoxyethyl), `NMA` (2'-O-[2-(methylamino)-2-oxoethyl]), `allyl` (2'-O-allyl), `AP` (2'-O-

aminopropyl), NH2 (2'-amino-2'-deoxyribose), UNA (unlocked, 2',3'-seco), GNA (glycol nucleic acid), LNA, ENA (2'-O,4'-C-ethylene), cEt (constrained ethyl), NP (3'-amino, for N3'->P5' phosphoramidate backbones).

Linkages: o/p (phosphodiester), s (phosphorothioate), u (PS stereochemistry variant), the alkyl phosphonates mp (methyl), prp (propyl), ibu (isobutyl), chx (cyclohexyl) and mop (methoxypropyl), pace/tpace (phosphonoacetate and its thio analog), msp (mesyl phosphoramidate), and pgo/tmg (the two phosphoryl guanidine variants – they differ by 2.0157 Da per bond, so check which one you have).

Conjugates: 5'/3'-phosphate caps, GalNAc (mono and trivalent cluster), cholesterol, C6/C12 amino linkers, TEG, FAM, Cy3, the fatty acids myristoyl/palmitoyl/stearoyl/docosanoyl, plus the BioPharma Finder 5.2 terminal-modification set (biotin, cAG/cAU/ARCA/mCAP cap analogs, triantennary GalNAc).

inst/help/MODIFICATIONS.md – the Modifications tab of the dashboard's Help panel – lists every one of these with its residue formula and its per-position mass difference from the parent chemistry, and explains how to add a modification the dictionary does not have.

Entries flagged verify = TRUE (e.g. cEt, LNA, ENA, msp, pgo, GalNAc3, dye conjugates) carry best-estimate formulas – confirm them against a known mass before relying on them, or override them (next section).

Custom chemistry overrides

Any code can be added or replaced with build_dictionary(). Each override is a named list carrying a formula (named vector or string) and an optional name; conjugates also take an attach mode (replace_H, replace_OH, or add).

```
custom_dict <- build_dictionary(list(
  # A new base code
  X = list(formula = c(C = 6, H = 7, N = 5, O = 1),
           name = "my modified base X"),
  # A new sugar code, formula as string
  Q = list(formula = "C10H20O7", name = "my 2'-mod sugar Q"),
  # Replace a best-estimate entry with a confirmed formula
  cEt = list(formula = "C7H12O4", name = "constrained ethyl (confirmed)")
))
format_formula(custom_dict[["Q"]]$formula)
#> [1] ""
```

Pass dict = custom_dict to any downstream function, and use the new codes in sequences exactly like standard ones (Gm-sXm-sQm-...). The pipeline trusts custom formulas as given – there is no external validation gate – so when a reference mass for the full-length oligo is available, compare it against the computed parent mass to catch transcription errors.

The formula-engine self-test

validate_reference() re-derives the molecular formulas of nusinersen and inotersen from their sequences and chemistry, and compares each against the free-acid formula on its approved product labelling. Between them they exercise MOE and DNA sugars, 5-methylcytosine and 5-methyluracil bases, and a fully phosphorothioate backbone. It is a regression check on the formula engine (run it after editing the dictionary), not a validation of whatever sequence you are analyzing:

```
v <- validate_reference(verbose = FALSE)
v$ok                        # TRUE only if both formulas reproduce exactly
#> [1] TRUE
vapply(v$all, function(x) x$formula, "")
#>              nusinersen               inotersen
#> "C234H340N61O128P17S17" "C230H318N69O121P19S19"
vapply(v$all, function(x) round(x$ppm, 6), 0)   # deviation, ppm
```

```
#> nusinersen  inotersen
#>          0          0
```

Metabolite library generation

`generate_metabolites()` builds the library from a parsed spec:

```
mets <- generate_metabolites(spec, opts = list(
  oligo_name   = "demo",
  max_3p       = 5,      # 3' exonuclease ladder depth
  max_5p       = 5,      # 5' exonuclease ladder depth
  endo         = TRUE,   # endonuclease internal fragments
  endo_sites   = "all",  # or "gap" to cleave only in the DNA gap
  min_frag_len = 3       # discard fragments shorter than this
))
head(metabolite_table(mets), 8)
#>    id        name   kind  n n_ps n_po            bases           modification
#> 1 M01        demo parent 16   15    0 GTCTCTCTCTTCTCTG                 parent
#> 2 M02 demo 3' N-1 exo_3p 15   14    0  GTCTCTCTCTTCTCT 3' exonuclease (-1 nt)
#> 3 M03 demo 3' N-2 exo_3p 14   13    0   GTCTCTCTCTTCTC 3' exonuclease (-2 nt)
#> 4 M04 demo 3' N-3 exo_3p 13   12    0    GTCTCTCTCTTCT 3' exonuclease (-3 nt)
#> 5 M05 demo 3' N-4 exo_3p 12   11    0     GTCTCTCTCTTC 3' exonuclease (-4 nt)
#> 6 M06 demo 3' N-5 exo_3p 11   10    0      GTCTCTCTCTT 3' exonuclease (-5 nt)
#> 7 M07 demo 5' N-1 exo_5p 15   14    0  TCTCTCTCTTCTCTG 5' exonuclease (-1 nt)
#> 8 M08 demo 5' N-2 exo_5p 14   13    0   CTCTCTCTTCTCTG 5' exonuclease (-2 nt)
#>   site
#> 1
#> 2    1
#> 3    2
#> 4    3
#> 5    4
#> 6    5
#> 7    1
#> 8    2
```

Options worth knowing:

- **`max_3p` / `max_5p`** – ladder depth. For an n-mer, `n - 1` gives the complete series; the default (10) covers the metabolites actually observed in most in vivo studies.
- **`endo_sites = "gap"`** – for gapmer ASOs, endonucleases (S1, RNase H-adjacent activities) preferentially cleave within the DNA gap. Restricting cleavage to the gap keeps the library focused and the downstream envelope computation fast.
- **`min_frag_len`** – short fragments produce low-m/z, low-charge species that rarely survive LC-MS detection; 3 is a practical floor.

Each metabolite records its own bases/sugars/linkages (so conjugate loss on truncation is handled correctly), its kind (`parent`, `exo_3p`, `exo_5p`, `endo_5p`/`endo_3p`), PS/PO linkage counts, and a human-readable modification label.

Masses, envelopes, isotopes, and the PS oxidation series

Mass and formula

```
info <- metabolite_mass_info(mets[[1]])
info$formula_str
#> [1] "C164H221N44O97P15S15"
round(info$mono_mass, 6)
#> [1] 5302.559
round(info$avg_mass, 4)
#> [1] 5306.293
```

The formula is assembled as: sum of bases + sum of sugars + sum of internal linkages - (2n - 1) H_2O + conjugates, accounting for n glycosidic bonds and n - *1* internucleotide bonds. This convention is validated against the published molecular formulas of nusinersen and inotersen (0 ppm on both).

Charge envelope

Negative ESI, $[M - zH]^{z-}$:

```
charge_envelope(info$mono_mass, z_range = 4:8)
#>   z        mz
#> 1 4 1324.6324
#> 2 5 1059.5045
#> 3 6  882.7525
#> 4 7  756.5011
#> 5 8  661.8126
```

`h_offset` shifts the neutral mass before charging. Leave it at 0 for standard mass spectrometry; a non-zero value exists only to reproduce legacy reference workbooks that computed their envelopes from an offset neutral mass.

Adduct satellites replace one H with a cation; the shifts are exposed via `adduct_shift()`:

```
sapply(c("Na", "K", "NH4"), adduct_shift)
#>       Na        K      NH4
#> 21.98194 37.95588 17.02655
```

Isotope patterns

`isotope_pattern()` returns the top-N isotopologues for a formula. With `enviPat` installed it uses enviPat's convolution; otherwise a built-in engine takes over (identical selection logic, slightly coarser peak merging). Both paths are memoized per formula, so repeated calls across charge states are effectively free.

```
pat <- isotope_pattern(info$formula_vec, n_top = 5, use_envipat = FALSE)
pat$mass <- round(pat$mass, 4)
pat
#>        mass  abundance
#> 4  5303.565 0.09016527
#> 19 5304.568 0.07899144
#> 28 5305.561 0.06121447
#> 74 5306.564 0.05362840
#> 1  5302.561 0.05083229
```

`isotope_mz_cluster()` converts a pattern into per-charge-state m/z values (the monoisotopic peak is always retained even when it is not among the most abundant), and `compute_envelope()` produces the full tidy table – one row per (oxidation level, charge state, isotope) – that feeds the workbook's Charge Envelopes sheet:

```
env <- compute_envelope(mets[[1]], z_range = 5:6, n_iso = 3,
                        max_oxid = 1, use_envipat = FALSE)
```

```
env$mz <- round(env$mz, 4)
head(env[, c("met_id", "k_oxid", "z", "iso", "mz", "abundance")], 6)
#>   met_id k_oxid z iso        mz  abundance
#> 1    M01      0 5   0 1059.5050 0.05083229
#> 2    M01      0 5   1 1059.7057 0.09016527
#> 3    M01      0 5   2 1059.9064 0.07899144
#> 4    M01      0 6   0  882.7530 0.05083229
#> 5    M01      0 6   1  882.9202 0.09016527
#> 6    M01      0 6   2  883.0874 0.07899144
```

PS oxidation series

Each PS-to-PO desulfurization replaces one S with one O (-15.977 Da). `ps_oxidation_series()` walks the ladder from 0 to `min(n_PS, max_oxid)` events:

```
ser <- ps_oxidation_series(mets[[1]], max_oxid = 3)
ser$mono_mass <- round(ser$mono_mass, 4)
ser[, c("k", "mono_mass", "formula_str")]
#>   k mono_mass           formula_str
#> 1 0  5302.559  C164H221N44O97P15S15
#> 2 1  5286.582  C164H221N44O98P15S14
#> 3 2  5270.605  C164H221N44O99P15S13
#> 4 3  5254.627 C164H221N44O100P15S12
```

Heavily oxidized species are low-abundance in practice; the default cap of 6 events covers what is typically observable.

MS/MS fragment ions

`generate_fragments()` produces McLuckey terminal ions – a, a-B, b, b-B, c, d from the 5' side and w, x, y, z from the 3' side – and `generate_internal_fragments()` adds internal (double-cleavage) products:

```
frags <- generate_fragments(mets[[1]])
length(frags)
#> [1] 120
ft <- fragment_table(frags)
head(ft[ft$z == 1, c("ion_type", "direction", "cleavage_site",
                     "frag_length", "mz")], 8)
#>    ion_type direction cleavage_site frag_length        mz
#> 1         a        5'             1           1  280.1051
#> 3       a-B        5'             1           1  147.0663
#> 5         b        5'             1           1  296.1000
#> 7       b-B        5'             1           1  163.0612
#> 9         c        5'             1           1  391.0357
#> 11        w        3'             1          15 5038.4497
#> 13        x        3'             1          15 5021.4469
#> 15        y        3'             1          15 4926.5113
```

For PS backbones, `check_ps_diagnostic()` looks for the low-mass PS diagnostic ions (94.9362 / 192.9660 series) in an MS2 spectrum, and `confirmation_score()` combines fragment coverage, diagnostic ions, and mass accuracy into a 0-100 confirmation score per metabolite (following the FMVS approach).

Outputs: workbook, report, acquisition lists

Excel workbook

`build_workbook()` writes the seven-sheet library workbook: Summary, Metabolite Library, Charge Envelopes, PS Oxidation Series, Fragment Ions, PRM Inclusion List, and MS Matching.

```
opts <- list(z_range = 3:12, n_iso = 5, max_oxid = 6, h_offset = 0,
             use_envipat = TRUE, include_internal = TRUE,
             max_3p = 10, max_5p = 10, endo = TRUE,
             adducts = c("H", "Na", "K", "NH4"), ppm_tol = 10)

wb_path <- build_workbook(spec, mets, opts = opts,
                          output_file = "demo_library.xlsx",
                          output_dir  = ".")
```

The Charge Envelopes sheet is the most computation-heavy (one isotope pattern per metabolite per oxidation level); a live console progress bar with an adaptive ETA tracks it. Sheets are assembled in memory and written in bulk, so the Excel serialization itself takes seconds even for libraries with tens of thousands of envelope rows.

HTML/PDF report

`build_report()` renders a standalone report (requires `rmarkdown`) with five summary plots – charge envelope, isotope pattern, truncation series, oxidation series, and fragment map. The individual `plot_*()` functions are exported too, so any plot can be regenerated or restyled on its own:

```
report <- build_report(spec, mets, opts = opts,
                       output_file = "demo_report",
                       output_format = "html",   # or "pdf"
                       plot_dir = ".")
```

Orbitrap Exploris acquisition method export

Three targeted-acquisition tables are exported in the exact column layout of the Orbitrap Exploris Method Editor's Targeted Mass filter (Mass List Type "m/z & z", Time Mode "Start/End Time"):

```
ms1 <- thermo_ms1_inclusion_list(mets[1:3], z_range = 5:8,
                                 max_oxid = 1, max_targets = 500)
head(ms1, 4)
#>   Compound Formula Adduct       m/z z Intensity Threshold t start (min)
#> 1     demo                1059.5045 5                                 0
#> 2     demo                 882.7525 6                                 0
#> 3     demo                 756.5011 7                                 0
#> 4     demo                 661.8126 8                                 0
#>   t stop (min) HCD Collision Energies (%) Maximum Injection Time (ms)
#> 1           30
#> 2           30
#> 3           30
#> 4           30
```

- **`thermo_ms1_inclusion_list()`** – every (metabolite, oxidation level, charge state) combination; import into a Full Scan experiment's Targeted Mass filter.
- **`thermo_ms2_prm_target_list()`** – the same list narrowed to a smaller charge range and tagged with an HCD collision energy; import as the target list of a Targeted MS2 (PRM) scan. The default NCE (20%) is a starting point for PS oligonucleotide backbones, not a validated parameter – optimize per

method.
- **`ms2_fragment_reference()`** – theoretical fragment ions per metabolite. This is *not* a Method Editor import (PRM records the full fragment spectrum); it is the lookup table for interpreting the acquired spectra afterward.

Both list functions cap output rows (`max_targets`) – the Method Editor accepts up to 150,000 rows, but PRM duty cycle degrades long before that.

Spectral libraries (MGF / MSP)

The same theoretical library also exports as MGF and MSP, the formats spectral-library tools read (MS-DIAL, mzVault/Compound Discoverer, MZmine, matchms, SIRIUS, GNPS).

```
export_spectral_libraries(mets, dict, out_dir = "results",
                          prefix = "my_oligo")
#> results/my_oligo_MS1_library.mgf  (+ .msp)
#> results/my_oligo_MS2_library.mgf  (+ .msp)
```

`build_ms1_library()` emits one spectrum per (metabolite, PS-oxidation level, charge state), whose peaks are the theoretical isotope cluster – real relative abundances, annotated with the true isotopologue offset (`M+0`, `M+1`, ...) and anchored on the m/z computed from the formula rather than on the isotope engine's binned masses.

`build_ms2_library()` emits one spectrum per (metabolite, precursor charge state), holding the McLuckey fragment ions at the requested fragment charges, annotated `w4^2-`, `a-B5^1-`, `w-a(3,8)^1-` in the MSP.

There is no fragment-intensity model in this pipeline, so **every MS2 peak is written at a flat intensity of 100**. Match on m/z; an intensity-weighted dot-product or cosine score against these libraries reduces to a function of peak count and means nothing. The MS1 libraries carry genuine isotope abundances and are not affected.

Matching against acquired MS data

Point the pipeline at an mzML/mzXML file (or a two-column m/z-intensity peak list) to annotate the library against real data:

```
ms   <- parse_mzml("data.mzML")            # MS1 + MS2 spectra
feat <- extract_ms1_features(ms$ms1)
hits <- match_ms1(feat, mets, ppm_tol = 10,
                  z_range = 3:12, max_oxid = 6,
                  adducts = c("H", "Na", "K", "NH4"))
ann  <- annotate_metabolites(hits, ms, mets)   # adds MS2 confirmation
```

Matching proceeds in the order established by the eluforsen and FMVS studies: (1) match each metabolite's theoretical m/z values across charge states, adducts, and oxidation levels at the given ppm tolerance; (2) score isotope-pattern fit (cosine similarity) and charge-envelope consistency; (3) for hits with associated MS2 spectra, confirm via McLuckey fragment matching and PS diagnostic ions.

Vendor raw files (Thermo `.raw`, Waters `.raw`, SCIEX `.wiff`) are converted automatically when ProteoWizard `msconvert` is on the PATH. mzML binary data (base64 + zlib) is decoded natively in R.

Batch processing and statistical comparison

The single-file matching above scales to a many-sample experiment via a bundled Python pipeline (`inst/python/oligomet_deconv/`, requires Python 3.9+ with `inst/python/requirements.txt` installed) that streams each mzML/mzXML file and detects chromatographic features by grouping observed

m/z into charge-state envelopes – the same neutral-mass agreement check the R side already uses for `envelope_consistency()`, `M = z * (mz - proton)`, rather than a peptide isotope/averagine model, since that doesn't describe oligonucleotides. One worker process per file, so a batch scales with core count; a corrupt file is isolated and logged, not fatal to the run.

```
source("R/batch_ms_processing.R"); source("R/statistics.R")

deconv   <- run_batch_deconvolution(list.files("raw_data", pattern = "\\.mzML$", full.names = TRUE))
features <- read_batch_features(deconv$features_path)
batch    <- annotate_metabolites_batch(mets, features, dict = dict)   # matches + retains unmatched pea

abund <- build_abundance_matrix(batch$ms1_matches)
long  <- abundance_long(abund, data.frame(
  sample = c("ctrl_1", "ctrl_2", "treat_1", "treat_2"),
  group  = c("control", "control", "treated", "treated")))
compare_two_groups(long, "control", "treated")   # Welch t-test + BH-adjusted p-values
```

R matches the resulting features against the theoretical library by reusing `match_ms1()` unchanged; anything that matches no candidate within tolerance is retained (not discarded) as an "unidentified peaks" table. When MS2 confirmation is enabled, R first computes a *targeted* watch-list of theoretical precursor m/z (reusing `prm_inclusion_list()`) and passes it to the Python pipeline, which captures MS2 spectra only for scans near those candidates in the same streaming pass – cheap on top of the deconvolution itself. R then confirms each matched hit against the theoretical fragment library with the existing `confirm_metabolite()` machinery, unchanged.

`compare_two_groups()` (Welch t-test), `compare_multi_groups()` (one-way ANOVA + Tukey HSD), and `compare_time_series()` (linear trend) all report Benjamini-Hochberg-adjusted p-values and flag metabolites with too few replicates rather than erroring.

Bundled example

A ready-to-run example ships with the package – 6 synthetic mzML files (3 "control" + 3 "treated" replicates of the inotersen reference sequence, plus a contaminant trace that matches no theoretical metabolite) under `system.file("extdata", "batch_example", package = "OligoMetProfiler")`. `Rscript inst/examples/run_batch_example.R` from a repository checkout runs the whole thing end to end with no editing; the same data is used live below when Python is available on the machine building this vignette.

```
example_dir <- system.file("extdata", "batch_example", package = "OligoMetProfiler")
batch_files <- list.files(example_dir, pattern = "\\.mzML$", full.names = TRUE)
sample_meta <- read.csv(file.path(example_dir, "sample_meta.csv"), stringsAsFactors = FALSE)

ex_spec <- parse_input(INOTERSEN_TRIPLET)
ex_mets <- generate_metabolites(ex_spec, opts = list(oligo_name = "inotersen_example",
                                                     max_3p = 3, max_5p = 3, endo = FALSE))

deconv   <- run_batch_deconvolution(batch_files, n_workers = 2)
features <- read_batch_features(deconv$features_path)
batch    <- annotate_metabolites_batch(ex_mets, features, ppm_tol = 10,
                                       z_range = 3:12, adducts = "H", max_oxid = 0, n_iso = 0)

cat("MS1 matches:", nrow(batch$ms1_matches),
    " | unidentified peaks retained:", nrow(batch$unmatched), "\n")
#> MS1 matches: 24  | unidentified peaks retained: 108

abund <- build_abundance_matrix(batch$ms1_matches)
```

```
long  <- abundance_long(abund, sample_meta)
compare_two_groups(long, "control", "treated")[, c("met_id", "met_name", "mean_a", "mean_b", "log2fc",
#>   met_id          met_name   mean_a   mean_b   log2fc        p_adj
#> 1    M01 inotersen_example 91886.56 289098.1 1.653633 1.658229e-06
```

The Shiny dashboard and CLI drivers

Three ready-made entry points wrap the same engine. The dashboard ships inside the installed package; the CLI drivers live in the repository root:

- **`run_app()`** – a Shiny dashboard with sequence input (triplet, or the three-field manual entry described above), an editable custom-chemistry table, full parameter control, optional MS upload, a four-plot summary, and workbook/report/PRM/acquisition-list/ spectral-library downloads. Launch with `OligoMetProfiler::run_app()` from an installed package, or `shiny::runApp(".")` from a clone of the repository.
- **`run_custom_oligo.R`** – the primary CLI template: copy it, edit the CONFIG block (sequence, overrides, parameters), and `Rscript run_custom_oligo.R`.
- **`run_batch_ms.R`** – the batch/multi-sample counterpart: same CONFIG-block pattern, but for a directory of raw files plus a sample metadata table (group or timepoint), running the parallel Python deconvolution pipeline and the statistics suite described above.

For a formula-engine regression check from the command line, run the validation scripts in `tests/` (e.g. `Rscript tests/test_mass_isotope.R`) or call `validate_reference()` directly from R.

Performance notes

- The **Charge Envelopes** computation dominates runtime: one isotope pattern per metabolite per PS-oxidation level. Isotope patterns are memoized per formula (both the enviPat and built-in engines), so charge states and adducts add no repeated cost.
- To speed up a long run: disable endonuclease fragments, lower `max_oxid`, or narrow `z_range` – each directly reduces the number of isotope-pattern calls.
- Workbook sheets are written in bulk (one write per sheet, styling restricted to headers and a few accent cells), so `saveWorkbook()` stays fast regardless of library size.
- `enviPat` prints a harmless note about the electron mass on some systems; it is cosmetic. Set `use_envipat = FALSE` to use the built-in engine instead.

Caveats

- Dictionary entries flagged `verify = TRUE` (cEt, LNA, GalNAc3, dye and BPF terminal modifications) are best-estimate formulas – confirm against a known mass before quantitative use.
- Custom overrides are trusted as given; validate the parent mass against a reference when one exists.
- The `h_offset = 3.0046` convention exists only to reproduce legacy reference workbooks that used an offset neutral mass; use 0 for standard work.
- The repository's `tests/` scripts are console-output validation scripts, not asserted unit tests.

Session info

```
sessionInfo()
#> R version 4.5.3 (2026-03-11)
#> Platform: x86_64-apple-darwin20
#> Running under: macOS Ventura 13.7.8
```

```
#>
#> Matrix products: default
#> BLAS:   /Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/lib/libRlapack.dylib;  LAPACK
#>
#> locale:
#> [1] en_US/en_US/en_US/C/en_US/en_US
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] OligoMetProfiler_1.0.0
#>
#> loaded via a namespace (and not attached):
#>  [1] compiler_4.5.3  fastmap_1.2.0   cli_3.6.6       tools_4.5.3
#>  [5] htmltools_0.5.9 otel_0.2.0      yaml_2.3.12     rmarkdown_2.31
#>  [9] knitr_1.51      xfun_0.60       digest_0.6.39   rlang_1.3.0
#> [13] evaluate_1.0.5
```

References

Methodology

1. Lippens J.L. *et al.* SynONIM: a Synthetic Oligonucleotide Nomenclature for Impurities and Modifications. *J Am Soc Mass Spectrom* (2024).
2. Kim J. *et al.* Metabolite Profiling of the Antisense Oligonucleotide Eluforsen Using Liquid Chromatography-Mass Spectrometry. *Mol Ther Nucleic Acids* (2019).
3. Liu R. *et al.* OligoDistiller: untargeted profiling of therapeutic oligonucleotide drugs and metabolites. *Anal Chem* (2025).
4. Ye X. *et al.* Automatic metabolite identification of antisense oligonucleotides by full MS variable scanning. *J Chromatogr B* (2025).
5. McLuckey S.A., Van Berkel G.J., Glish G.L. Tandem mass spectrometry of small, multiply charged oligonucleotides. *J Am Soc Mass Spectrom* 3:60-70 (1992).

Reference drugs and modality classification

6. Takakusa H., Iwazaki N., Nishikawa M., Yoshida T., Obika S., Inoue T. Drug Metabolism and Pharmacokinetics of Antisense Oligonucleotide Therapeutics: Typical Profiles, Evaluation Approaches, and Points to Consider Compared with Small Molecule Drugs. *Nucleic Acid Therapeutics* 33(2):83-94 (2023). doi:10.1089/nat.2022.0054 – source of the modality classification (Table 1) used for the four bundled examples, and of the class-specific metabolism routes (endonuclease-first for gapmers, exonuclease for SSOs).
7. Egli M., Manoharan M. Chemistry, structure and function of approved oligonucleotide therapeutics. *Nucleic Acids Research* 51(6):2529-2573 (2023). doi:10.1093/nar/gkad067 – chemistry and structural review of the approved oligonucleotide drugs.
8. SPINRAZA (nusinersen) prescribing information, Biogen – sequence, 2'-MOE/PS chemistry, and free-acid molecular formula $C_{234}H_{340}N_{61}O_{128}P_{17}S_{17}$.
9. TEGSEDI (inotersen) prescribing information, Akcea/Ionis – 5-10-5 MOE gapmer design, TTR 3'UTR target sequence (bases 618-637), and free-acid molecular formula $C_{230}H_{318}N_{69}O_{121}P_{19}S_{19}$ (average MW

7183.08).
10. ONPATTRO (patisiran) prescribing information, Alnylam – siRNA duplex sequences, 2'-OMe modification pattern, dTdT overhangs, and lipid-nanoparticle formulation.
11. GIVLAARI (givosiran) prescribing information, Alnylam – ESC-GalNAc siRNA sequences, 2'-F/2'-OMe alternation, terminal phosphorothioate placement, and the trivalent GalNAc (L96) 3' conjugate.

Instrument and software documentation

12. Thermo Fisher Scientific. *BioPharma Finder 5.2 Oligonucleotide Analysis User Guide* – triplet sequence notation and terminal modification codes.
13. Thermo Fisher Scientific. *Orbitrap Exploris 120 Software Manual*, "Targeted Inclusion – Targeted Mass filter" – mass-list column layout used by the acquisition exports.

Author, disclosure and disclaimer

Nishikant Wase, PhD – author and developer; Research Scientist, Thermo Fisher Scientific (nishikant.wase@gmail.com). Designed, written and maintained by the author; released under the MIT licence.

Research use only. OligoMetProfiler is a research tool. It is not a medical device and is not intended or validated for diagnostic use, clinical decision-making, patient care, quality control release testing, or inclusion in a regulatory submission.

Everything it reports is a prediction. Libraries, formulas, masses, envelopes, isotope patterns, fragment ions, target lists and spectral libraries are computed from a chemistry dictionary and from assumptions about oligonucleotide metabolism and fragmentation – not measured. A mass match is a hypothesis, not an identification; confirm every assignment experimentally. Formulas flagged `verify = TRUE` are unverified best estimates, and the MS2 spectral libraries carry placeholder intensities.

No warranty, no liability. The software is provided "as is", without warranty of any kind, express or implied, under the MIT licence. In no event shall the author be liable for any claim, damages or other liability – including loss of data, wasted instrument time or materials, or an erroneous scientific conclusion – arising from or in connection with the software or its use. The user is solely responsible for verifying fitness for their purpose and for validating every result.

Affiliation and disclosure. The author is employed as a Research Scientist at Thermo Fisher Scientific, disclosed here because the package interoperates with Thermo Fisher products (BioPharma Finder notation, Orbitrap Exploris mass lists). OligoMetProfiler is an independent personal project, not a Thermo Fisher Scientific product, and has not been supplied, reviewed, supported, endorsed or approved by Thermo Fisher Scientific or any other company; all interoperability is implemented from publicly documented formats only. Product and drug names are used for identification and interoperability only and remain the trademarks of their owners. Views and outputs are the author's own.

```
oligomet_about()   # prints this statement in the console
```

The full statement is in `DISCLAIMER.md`.